EE5203 L08 Lab Guide

PWM and input capture — generate a signal, then measure it back - Basri Erdogan

Mission: Generate PWM on PA5 and dim LD2 without a single `HAL_Delay()`. Then close the loop: one jumper carries PA5 to PB6, TIM4 captures the signal, and you prove the measured frequency and duty match what you configured.

Prepare before class

- ☐ Re-read the theory slides through "From period to frequency."
- ☐ Bring your working L07 tick project (TIM2, 1 ms, `g_ms`).
- ☐ Bring **one jumper wire**. On the Arduino header: PA5 = **D13**, PB6 = **D10**.
- ☐ Compute: at `PSC = 15`, `ARR = 999` from 16 MHz — what PWM frequency, and what `CCR` gives 35 %?

Learning objectives

By checkoff time, you can:

1. configure a PWM channel from the real 16 MHz clock and steer its duty live;
2. hand a pin over to a timer and prove the handover in `MODER`;
3. configure direct and indirect capture channels and read hardware timestamps;
4. turn timestamps into period, frequency, and duty — wrap-safe.

Hardware and files

Item	Required value
Board	Nucleo-F401RE — LD2 **is** TIM2_CH1 (PA5): no wiring for Part 1
Generator	TIM2 CH1, `PSC` 15, `ARR` 999 → 1 kHz, µs units
Analyzer	TIM4 CH1/CH2 on PB6, `PSC` 15 (1 µs tick), `ARR` 65535 (free-run)
Wiring (Part 2)	one jumper: PA5 (D13) → PB6 (D10)

Configure in CubeMX

From a copy of the L07 project, saved as `L08_PWM_Capture`:

1. `PA5` → `TIM2_CH1`; **Timers → TIM2**: Channel 1 = `PWM Generation CH1`, `PSC` 15, `ARR` 999.
2. `PB6` → `TIM4_CH1`; **Timers → TIM4**: Channel 1 = `Input Capture direct mode`, polarity **rising**; Channel 2 = `Input Capture indirect mode`, polarity **falling**; `PSC` 15, `ARR` 65535.
3. **NVIC**: enable **TIM4 global interrupt**.
4. Generate, then find both AF assignments in the generated GPIO init before writing any code of your own.

Checkpoint A - A dimmed LED

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 350);   // 35 %
```

Evidence for Checkpoint A: LD2 visibly dimmer than full-on. While paused, write `700` straight into `TIM2->CCR1` from the SFR view and watch the brightness step — no code changed, no reflash. Then show `MODER` bits 11:10 = `10`: the pin now belongs to the timer.

Checkpoint B - The breathing LED

No `HAL_Delay()` — schedule from the L07 tick. Every 10 ms, step the duty:

```
if ((g_ms - t_fade) >= 10U) {
    t_fade = g_ms;
    duty += dir * 5;
    if (duty >= 999) { duty = 999; dir = -1; }
    if (duty <= 0)   { duty = 0;   dir = +1; }
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, duty);
}
```

Evidence for Checkpoint B: smooth breathing, no `HAL_Delay()` in `main.c`. Predict then check: what does a step of 20 instead of 5 do to the breathing period?

Checkpoint C - Captures arrive

Fit the jumper (D13 → D10) and start the analyzer:

```
HAL_TIM_IC_Start_IT(&htim4, TIM_CHANNEL_1);   // rising, direct
HAL_TIM_IC_Start_IT(&htim4, TIM_CHANNEL_2);   // falling, indirect
```

Evidence for Checkpoint C: a breakpoint in `HAL_TIM_IC_CaptureCallback` is hit, and `TIM4->CCR1` **changes by itself** between pauses — hardware is timestamping with no help from your code.

Checkpoint D - Measure period and duty

In the capture callback:

```
if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) {          // rising
    uint16_t now = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
    period_us = (uint16_t)(now - last);                   // wrap-safe
    last = now;
} else if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_2) {   // falling
    high_us = (uint16_t)(HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_2) - last);
}
```

Freeze the duty at 350 first. **Predict `period_us` and `high_us` before you look**, then compare against the generator's own `ARR` and `CCR1`.

Evidence for Checkpoint D: `period_us` 1000 and duty 35 %. Then change the generator's `ARR` to `1999` and watch the measurement follow to 2000 — and re-enable the breathing sweep and watch the measured duty track it live.

Individual extension

Pick one, predict first:

- **Overcapture:** pause through several PWM periods, resume, and find `CC1OF` set in `TIM4->SR`. Explain precisely what was lost.
- **Polarity flip:** clear/set `CC1P` in `TIM4->CCER` from the SFR view and explain what the *period* measurement does — and does not — change.
- **Servo (hardware permitting):** TIM3 CH1 on PA6 (D12), `PSC` 15, `ARR` 19999 → 50 Hz. At a 1 µs tick `CCR` **is** the pulse width; clamp to the 1000–2000 µs contract and command 0°, 90°, 180°.

Acceptance checklist

- ☐ `.ioc`: TIM2 CH1 PWM `PSC` 15 / `ARR` 999; TIM4 CH1 direct-rising, CH2 indirect-falling, free-run; TIM4 NVIC enabled.
- ☐ Live duty change demonstrated from the SFR view.
- ☐ `MODER` handover shown for PA5.
- ☐ No `HAL_Delay()` anywhere.
- ☐ Callback checks the channel; subtractions cast to `uint16_t`.
- ☐ Measured period and duty match the generator's registers, and track a live change.
- ☐ One extension demonstrated with its SFR evidence.

Individual oral check

- Why does `CCR = ARR` give 99.9 % rather than 100 %?
- Who writes `TIM4->CCR1`, and at exactly what instant?
- Why must the subtraction be cast to `uint16_t`?
- Why is CH2 configured *indirect*? What would *direct* listen to?
- Your period reads 3036 after captures 64000 and 1500 — explain.
- Why can the 1 kHz LED and a 50 Hz servo not share one timer?

Submit

Submit `main.c` and one SFR-view screenshot showing `TIM2->CCR1`, `TIM4->CCR1`, `TIM4->CCR2`, and `TIM4->SR` with a capture flag set — plus your predicted-vs-measured table for Checkpoint D.

If you are stuck

Generating: **no dimming** — is `PA5` really `TIM2_CH1` in the `.ioc`, and did `PWM_Start` run? `MODER` still `01` means the handover never happened. Measuring: jumper seated (D13 → D10)? Generator actually running? `Start_IT` called for **both** channels? Then: **period garbage** — check the cast and the polarity; **duty > 100 %** — CH2 is probably direct, or the edges are swapped; **`CCR1` frozen** — capture never started, or the TIM4 NVIC box is unticked.